

Tính đúng và chứng minh (Correctness and Proof)

PHỤ THUỘC A/01 cho vai trò của chứng minh trong bước 3; A/03 cho ngôn ngữ chỉ phí. Chương này là điều kiện cần cho C/15 (tham lam) và C/16 (quy hoạch động), nơi “có vẻ đúng” hỏng nhiều nhất.

TÓM TẮT

Chương này cấp cái thước thứ hai: *lời giải của tôi có đúng không*. Công cụ trung tâm là **bất biến** (*invariant*) — một mệnh đề luôn đúng tại một điểm cố định trong thuật toán. Bất biến có hai vai trò và vai trò thứ hai hay bị bỏ quên: nó vừa để chứng minh, vừa để *nghe ra* thuật toán. Chương đi qua bốn kiểu lập luận dùng nhiều nhất — bất biến vòng lặp, quy nạp, lập luận đổi chỗ cho tham lam, lập luận cắt-dán cho quy hoạch động — cộng với cách tìm phản ví dụ và cách chứng minh thuật toán *dùng*. Nếu chỉ nhớ một điều: viết một dòng bất biến trước mỗi vòng lặp là việc rẻ nhất bạn có thể làm để loại bỏ phần lớn lỗi lệch một đơn vị và lỗi khởi tạo.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bất biến vòng lặp (loop invariant)** — mệnh đề đúng ngay trước mỗi lần lặp và vẫn đúng sau mỗi lần lặp. Ba việc phải kiểm: khởi tạo đúng, được duy trì, và khi thoát thì cho ra kết luận cần.
- **quy nạp (induction)** — chứng minh mệnh đề đúng cho mọi n bằng cách chứng minh nó đúng cho trường hợp nhỏ nhất, rồi chứng minh “đúng tới $n - 1$ kéo theo đúng ở n ”.
- **đại lượng giảm (variant / measure)** — một số nguyên không âm giảm nghiêm ngặt sau mỗi vòng lặp. Dùng để chứng minh thuật toán *dùng*, thứ mà bất biến không đảm bảo.
- **lập luận đổi chỗ (exchange argument)** — kỹ thuật chứng minh tham lam: lấy một lời giải tối ưu bất kỳ, biến đổi nó từng bước cho giống lựa chọn tham lam mà không làm nó tệ đi.
- **tiền điều kiện / hậu điều kiện** — điều kiện phải đúng trước khi chạy một đoạn mã, và điều kiện được đảm bảo sau khi chạy xong. Ngôn ngữ của Hoare[1].
- **phản ví dụ** — một dữ liệu vào cụ thể làm thuật toán cho kết quả sai. Một phản ví dụ đủ để bác bỏ; một triệu ví dụ đúng không đủ để chứng minh.

CÂU HỎI MỞ ĐẦU

- Q1. Chạy đúng trên một nghìn bộ test ngẫu nhiên — vì sao vẫn chưa phải bằng chứng đúng, và loại lỗi nào sống sót qua kiểm thử ngẫu nhiên?
- Q2. Bất biến là công cụ chứng minh. Vì sao chương này lại nói nó cũng là công cụ *sinh* thuật toán?
- Q3. Vì sao chứng minh một thuật toán tham lam khó hơn hẳn chứng minh một thuật toán quy hoạch động, dù code tham lam ngắn hơn nhiều?
- Q4. Một thuật toán có thể luôn giữ đúng bất biến mà vẫn sai. Sai ở đâu, và cần thêm gì?
- Q5. Khi nào thì bỏ việc chứng minh là quyết định hợp lý, chứ không phải lười?

Có một trải nghiệm mà ai luyện thuật toán cũng gặp: bạn viết xong, chạy thử vài ví dụ, thấy đúng, nộp bài, và nhận về “sai ở test 47”. Bạn không biết test 47 là gì. Bạn nhìn code, mọi dòng đều hợp lý. Và bạn bắt đầu sửa mò — đổi $<$ thành $\leq$, đổi khởi tạo, nộp lại. Đó là trạng thái tệ nhất trong việc lập trình: sửa mà không hiểu, và không biết khi nào thì xong.

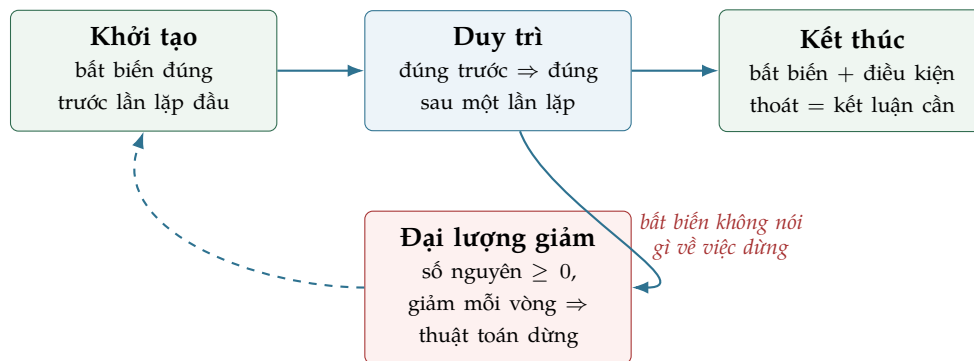
Chương này nói về cách thoát khỏi trạng thái đó, và câu trả lời không phải “kiểm thử nhiều hơn”. Dijkstra đã nói câu chính xác nhất về giới hạn của kiểm thử: nó cho thấy sự *hiện diện* của lỗi, không bao giờ cho thấy sự *vắng mặt*[2]. Muốn biết chương trình đúng, phải có một lập luận — và lập luận đó không nhất thiết phải hình thức, phần lớn thời gian nó chỉ là một hai câu viết ra cho rõ ràng.

Điều bất ngờ, và là luận điểm chính của chương: lập luận đó không phải việc làm *sau* khi có thuật toán. Làm đúng cách, nó chính là việc giúp bạn *tìm ra* thuật toán.

1.1 Bất biến vòng lặp: ba việc phải kiểm

Bất biến là một mệnh đề đúng tại một điểm cố định trong vòng lặp — thường là ngay trước mỗi lần lặp. Để dùng nó chứng minh một vòng lặp, cần kiểm ba việc, đúng như cấu trúc của phép quy nạp.

Khởi tạo: bất biến đúng trước lần lặp đầu tiên. *Duy trì*: nếu đúng trước một lần lặp thì vẫn đúng trước lần lặp kế tiếp. *Kết thúc*: khi vòng lặp thoát, bất biến cộng với điều kiện thoát cho ra đúng điều ta cần.



Hình 1.1. Bộ khung chứng minh một vòng lặp. Ba ô trên là bất biến; ô dưới là thứ thường bị quên: bất biến chỉ nói “nếu dừng thì đúng”, nó không hứa vòng lặp sẽ dừng. Muốn có tính dừng cần thêm một đại lượng nguyên không âm giảm nghiêm ngặt mỗi vòng.

Hãy làm một ví dụ đủ nhỏ để thấy hết. Tìm kiếm nhị phân trên mảng đã sắp tăng $a[0..n-1]$, tìm vị trí đầu tiên có $a[i] \geq x$. Bất biến chọn như sau: *mọi phần tử ở vị trí $< lo$ đều nhỏ hơn x , và mọi phần tử ở vị trí $\geq hi$ đều $\geq x$* . Nói cách khác, đáp án luôn nằm trong đoạn $[lo, hi]$.

Khởi tạo: đặt $lo = 0, hi = n$. Hai mệnh đề đúng một cách rỗng vì không có vị trí nào < 0 hay $\geq n$. *Duy trì*: lấy mid ở giữa; nếu $a[mid] < x$ thì mọi vị trí $\leq mid$ cũng nhỏ hơn x (vì mảng đã sắp), nên đặt $lo = mid + 1$ vẫn giữ bất biến; ngược lại đặt $hi = mid$. *Kết thúc*: khi $lo = hi$, bất biến nói mọi thứ bên trái nhỏ hơn x và mọi thứ từ lo trở đi $\geq x$ — đúng định nghĩa của vị trí cần tìm.

Điểm đáng chú ý: mọi câu hỏi cài đặt gây bối rối đều có câu trả lời rút ra từ bất biến, chứ không phải từ việc thử. hi khởi tạo bằng n hay $n - 1$? Bằng n , vì bất biến nói hi là vị trí đầu tiên đã biết là $\geq x$, mà có thể không có vị trí nào như vậy. Cập nhật $lo = mid + 1$ hay $lo = mid$? Bằng $mid + 1$, vì ta vừa chứng minh $a[mid] < x$ nên mid không thể là đáp án. Vòng lặp chạy khi $lo < hi$ hay $lo \leq hi$? Khi $lo < hi$, vì $lo = hi$ đã là kết luận.

CẦN NHỚ

Viết bất biến *trước* khi viết vòng lặp, dưới dạng một dòng chú thích bằng tiếng Việt. Đây là thói quen rẻ nhất và hiệu quả nhất trong cả quyển sách: nó biến các quyết định “< hay

<=" từ chỗ đoán mò thành chỗ suy ra được. Bentley[3] nhận xét rằng rất nhiều lập trình viên chuyên nghiệp viết sai tìm kiếm nhị phân khi phải viết từ đầu — gần như luôn vì họ viết code trước rồi mới nghĩ về ý nghĩa của các biến.

1.2 Tính dừng: thứ bất biến không hứa

Một điểm tinh tế mà người mới hay bỏ qua: bất biến chỉ chứng minh *nếu vòng lặp dừng thì kết quả đúng*. Nó hoàn toàn không nói vòng lặp sẽ dừng. Vòng lặp `while (true) {}` giữ mọi bất biến một cách hoàn hảo.

Công cụ cho tính dừng là *đại lượng giảm*: một số nguyên không âm, giảm ít nhất một đơn vị sau mỗi vòng. Vì số nguyên không âm không giảm được vô hạn lần, vòng lặp phải dừng. Với tìm kiếm nhị phân, đại lượng đó là $hi - lo$. Với thuật toán Euclid tính ước chung lớn nhất, là giá trị của số nhỏ hơn. Với thuật toán Ford–Fulkerson trên đồ thị có trọng số nguyên (chương C/18), là tổng luồng còn thiếu — và chính chỗ này lộ ra vì sao thuật toán đó có thể không dừng khi trọng số là số vô tỉ.

Trong đệ quy, đại lượng giảm chính là “kích thước bài toán con nhỏ hơn kích thước bài toán gốc”. Nghe hiển nhiên, nhưng đây đúng là chỗ hỏng của rất nhiều lời giải đệ quy sai: một nhánh nào đó gọi lại chính nó với cùng kích thước.

1.3 Quy nạp: khuôn của mọi chứng minh về cấu trúc đệ quy

Quy nạp và đệ quy là hai mặt của một đồng xu: đệ quy giải bài lớn bằng bài nhỏ, quy nạp chứng minh mệnh đề cho n bằng mệnh đề cho các giá trị nhỏ hơn. Vì vậy mọi thuật toán đệ quy đều được chứng minh tự nhiên bằng quy nạp, và hai phần khớp nhau từng dòng.

Jeff Erickson[4] có một cách diễn đạt rất hữu ích cho việc viết đệ quy: hãy giả định rằng mọi lời gọi đệ quy *đã* cho kết quả đúng — ông gọi đó là “bà tiên đệ quy” — và bạn chỉ cần lo đúng một việc: từ kết quả của các bài toán con, ghép ra kết quả của bài toán gốc. Đây không phải mẹo tâm lý mà chính là bước quy nạp, viết dưới dạng dễ dùng. Người mới hay tự làm khổ mình bằng cách cố hình dung toàn bộ cây đệ quy; điều đó vừa bất khả thi vừa không cần thiết.

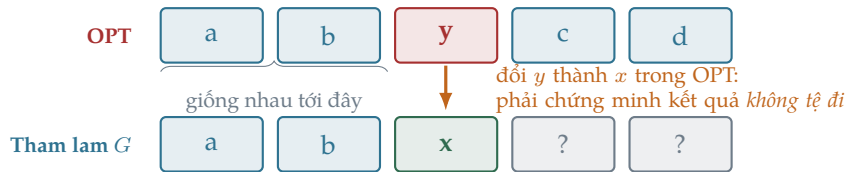
Hai biến thể đáng biết. *Quy nạp mạnh* giả định mệnh đề đúng cho *mọi* giá trị nhỏ hơn n , không chỉ $n - 1$ — cần thiết cho các thuật toán chia bài toán thành nhiều mảnh không đều. *Quy nạp cấu trúc* làm việc trên cây, đồ thị hoặc biểu thức thay vì trên số: chứng minh mệnh đề đúng ở lá, và đúng ở một nút nếu đúng ở mọi con của nó. Gần như mọi chứng minh về cây trong phần B đều theo khuôn này.

1.4 Lập luận đổi chỗ: vũ khí để chứng minh tham lam

Thuật toán tham lam là loại có khoảng cách lớn nhất giữa độ ngắn của code và độ khó của chứng minh. Code thường bốn dòng; chứng minh thì đòi hỏi thật sự, và nếu bỏ qua thì bạn sẽ nộp một lời giải sai với vẻ ngoài rất thuyết phục.

Khuôn chứng minh chuẩn là *lập luận đổi chỗ*: giả sử tồn tại một lời giải tối ưu OPT khác với lời giải tham lam G . Xét chỗ khác nhau *đầu tiên*. Chứng minh rằng ta có thể sửa OPT tại chỗ đó cho

giống G mà không làm nó tệ đi. Lặp lại, ta biến OPT thành G qua hữu hạn bước mà chất lượng không giảm — vậy G cũng tối ưu.



Hình 1.2. Sơ đồ lập luận đổi chỗ. Ta không cố chứng minh trực tiếp rằng tham lam tối ưu; ta chứng minh mọi lời giải tối ưu đều *biến đổi được* về lời giải tham lam mà không mất chất lượng. Toàn bộ công việc nằm ở mũi tên: chứng minh phép đổi tại vị trí khác nhau đầu tiên là hợp lệ và không làm tệ đi.

Ví dụ chuẩn: chọn nhiều nhất các cuộc họp không chồng lấn. Tham lam chọn cuộc họp *kết thúc sớm nhất*. Chứng minh: giả sử OPT chọn cuộc họp đầu tiên là y trong khi tham lam chọn x với thời điểm kết thúc $f(x) \leq f(y)$. Thay y bằng x trong OPT : mọi cuộc họp còn lại của OPT bắt đầu sau $f(y) \geq f(x)$, nên chúng vẫn không chồng lấn với x . Số cuộc họp không đổi, vậy lời giải mới vẫn tối ưu. Lặp lại cho vị trí thứ hai, thứ ba, và ta xong.

Một khuôn họ hàng là *tham lam luôn đi trước* (greedy stays ahead): chứng minh bằng quy nạp rằng sau k bước, lời giải tham lam tốt ít nhất bằng bất kỳ lời giải nào khác sau k bước. Với bài xếp lịch trên, sau k cuộc họp, thời điểm kết thúc của tham lam luôn sớm nhất có thể — và đó chính là lý do nó không bao giờ bị kẹt.

CẠM BÃY

Cạm bẫy chết người của tham lam: dùng “tôi thử vài ví dụ thấy đúng” làm chứng minh. Nhiều tiêu chí tham lam sai chỉ hổng ở những ví dụ khá đặc biệt. Ví dụ bài đổi tiền với các mệnh giá 1, 3, 4 và số tiền 6: tham lam lấy 4 rồi 1 rồi 1 (ba tờ), trong khi tối ưu là 3 + 3 (hai tờ). Nếu bạn chỉ thử với mệnh giá quen thuộc như 1, 5, 10 thì không bao giờ thấy lỗi. Quy tắc: hoặc chứng minh được bằng đổi chỗ, hoặc stress test với lời giải vét cạn (chương E/25); “thấy đúng” không nằm trong hai lựa chọn đó.

1.5 Lập luận cắt-dán: nền của quy hoạch động

Quy hoạch động đúng khi bài toán có *cấu trúc con tối ưu*: lời giải tối ưu của bài lớn chứa trong nó lời giải tối ưu của các bài con. Chứng minh chuẩn cho tính chất này là *cắt-dán*: giả sử lời giải tối ưu của bài lớn chứa một lời giải *không* tối ưu cho bài con nào đó; cắt phần đó ra, dán lời giải tối ưu của bài con vào, và ta được lời giải tốt hơn cho bài lớn — mâu thuẫn.

Nghe hình thức, nhưng nó có tác dụng thực hành rất cụ thể: khi cấu trúc con tối ưu *không* đúng, lập luận cắt-dán sẽ hổng ở một chỗ nhìn thấy được, và chỗ hổng đó cho bạn biết trạng thái đang thiếu chiều nào. Ví dụ kinh điển: đường đi *dài nhất* không có chu trình trong đồ thị tổng quát không có cấu trúc con tối ưu — ghép hai đường dài nhất có thể tạo ra đỉnh lặp. Nhận ra điều đó bằng cắt-dán nhanh hơn nhiều so với cài xong rồi mới thấy sai.

1.6 Tìm phản ví dụ: kỹ năng đối ngẫu với chứng minh

Chứng minh và phản ví dụ là hai mặt của cùng một việc, và người giải bài giỏi luôn làm cả hai xen kẽ: thử chứng minh vài phút, nếu chỗ nào không chứng minh được thì soi đúng chỗ đó để tìm phản ví dụ.

Ba nơi phản ví dụ hay ẩn nấp. *Trường hợp suy biến*: rỗng, một phần tử, mọi phần tử bằng nhau, đồ thị không liên thông, cạnh trọng số 0 hoặc âm. *Ranh giới của tiêu chí*: khi hai lựa chọn hoà nhau theo tiêu chí tham lam, cách phá hoà quyết định đúng sai. *Chỗ mà lập luận của bạn phải nói “hiển nhiên”*: mỗi lần bạn thấy mình viết chữ đó, hãy dừng lại kiểm tra — đó là nơi có mật độ lỗi cao nhất.

Và khi tìm bằng tay không ra, hãy để máy tìm: sinh dữ liệu nhỏ ngẫu nhiên, chạy song song lời giải nhanh và lời giải vét cạn, so kết quả. Kỹ thuật này — stress test — được nói kỹ ở chương E/25, và nó là cách tìm phản ví dụ hiệu quả nhất mà cũng ít tốn công nhất.

1.7 Chứng minh tới mức nào là đủ

Cần trung thực về liều lượng. Chứng minh hình thức đầy đủ cho mọi dòng code là điều không ai làm trong thực tế, kể cả người viết sách về chứng minh. Câu hỏi đúng không phải “có chứng minh hay không” mà là “chứng minh chặt tới mức nào cho phần nào”.

Có ba mức, và chọn mức là một quyết định kỹ thuật. *Mức một* — *một câu bằng lời*: dùng cho mọi vòng lặp, mọi lúc. Ví dụ “sau mỗi bước, best là đáp án tốt nhất trong $a[0..i]$ ”. Rẻ tới mức không có lý do gì để bỏ. *Mức hai* — *lập luận đầy đủ nhưng không hình thức*: dùng cho mỗi tiêu chí tham lam, mỗi công thức truy hồi quy hoạch động, mỗi chỗ bạn thấy “hình như đúng”. Đây là mức mà chương này chủ yếu dạy. *Mức ba* — *hình thức hoá*: dành cho những chỗ thật sự tinh vi, hiếm, và cho phần mềm mà lỗi có giá quá đắt.

Ngược lại, có những lúc bỏ chứng minh là hợp lý: trong kỳ thi khi bạn còn 10 phút và có thể stress test thay thế; khi thuật toán là một biến thể rất gần với một thuật toán đã được chứng minh; hoặc khi hậu quả của sai là nhỏ và phát hiện được ngay. Điều không hợp lý là bỏ chứng minh và bỏ luôn stress test, rồi tin vào cảm giác.

TRỰC GIÁC

Vì sao viết bất biến lại giúp *nghĩ ra* thuật toán chứ không chỉ kiểm chứng nó? Vì bất biến buộc bạn phát biểu “sau khi xử lý xong i phần tử đầu, tôi đang giữ đúng thông tin gì”. Câu hỏi đó chính là câu hỏi thiết kế trung tâm: nếu thông tin bạn giữ đủ để tính bước tiếp theo thì bạn có thuật toán; nếu chưa đủ thì bạn vừa phát hiện chính xác cái còn thiếu, và thứ còn thiếu đó thường trở thành một chiều mới của trạng thái. Đây là lý do người quen viết bất biến ít khi phải viết lại code từ đầu: họ phát hiện lỗ hổng ở giai đoạn suy nghĩ, chỗ mà sửa còn rẻ.

1.8 Máy trạng thái: một khung duy nhất cho ba loại bất biến

Chương này đã dùng bất biến ở ba vai trò trông rất khác nhau: bất biến vòng lặp để chứng minh một thuật toán đúng (mục 1), bất biến kiểu olympiad — chắn lẻ, tô màu — để chứng minh một cấu hình *không thể* đạt tới, và ở chương C/17 sẽ có thêm đồ thị trạng thái mà ta duyệt bằng tìm kiếm. Ba vai trò ấy là một, và Lehman, Leighton cùng Meyer[5] trình bày khung gộp chúng gọn hơn cả: **máy trạng thái** (*state machine*).

Máy trạng thái gồm ba thứ: một tập trạng thái, một trạng thái đầu, và một quan hệ chuyển. Một **bất biến bảo toàn** (*preserved invariant*) là một vị từ P trên trạng thái sao cho hễ $P(q)$ đúng và có bước chuyển $q \rightarrow r$ thì $P(r)$ cũng đúng. *Nguyên lý bất biến* phát biểu: nếu P đúng ở trạng thái đầu và được bảo toàn qua mọi bước chuyển, thì P đúng ở mọi trạng thái tới được. Nói cho đủ, đây chỉ là quy nạp theo số bước — nhưng viết ở dạng này thì dùng được ngay mà không phải dựng lại phép quy nạp mỗi lần.

Ví dụ nhỏ nhất trong sách của họ đáng nhớ vì nó gói cả ý tưởng trong ba dòng. Một robot xuất phát tại $(0, 0)$ trên lưới nguyên, mỗi bước đi chéo: $(m, n) \rightarrow (m \pm 1, n \pm 1)$. Nó có tới được $(1, 0)$ không? Xét vị từ “ $m + n$ chẵn”. Nó đúng ở trạng thái đầu, và mỗi bước làm tổng đổi đi 0, +2 hoặc -2 nên tính chẵn được giữ. Mà $(1, 0)$ có tổng lẻ. Vậy robot *không bao giờ* tới được $(1, 0)$ — và chú ý loại kết luận: đây là điều mà không lượng thời gian chạy tìm kiếm nào chứng minh được, vì tìm kiếm chỉ có thể chạy mãi mà không tìm thấy. Bài Fifteen Puzzle trong cùng chương của họ là bản lớn hơn của đúng lập luận này.

Khung này đáng nhớ vì nó gộp ba việc thành một, và mỗi lần gộp là một lần bạn phải nhớ ít đi.

Vòng lặp là một máy trạng thái mà trạng thái là bộ giá trị các biến và bước chuyển là một lần chạy thân vòng lặp. Bất biến vòng lặp chính là bất biến bảo toàn, và “ba việc phải kiểm” ở mục 1 — đúng lúc đầu, được giữ qua mỗi vòng, kết luận được điều cần khi thoát — đúng là ba phần của nguyên lý trên.

Bài toán tìm kiếm trên không gian trạng thái cũng vậy, chỉ khác là ta duyệt thay vì lập luận. Và đây là chỗ khung này trả công nhiều nhất trong lúc thi: trước khi viết duyệt rộng trên hàng triệu trạng thái, hãy dành một phút hỏi có bất biến nào không. Nếu trạng thái đích vi phạm một bất biến thì khỏi chạy — đáp án là “không thể”. Nếu bất biến chia không gian thành nhiều lớp thì bạn chỉ phải duyệt lớp chứa trạng thái đầu, và không gian bài toán nhỏ đi theo đúng số lớp.

Còn tính dừng thì bất biến không nói gì, đúng như mục 2 đã cảnh báo. Máy trạng thái làm rõ chỗ khác nhau: bất biến là phát biểu về *tập trạng thái tới được*, còn đại lượng giảm là phát biểu về *độ dài của đường đi*. Hai câu hỏi khác nhau thì cần hai công cụ khác nhau, và lần chúng là lỗi phổ biến khi người ta tưởng đã chứng minh xong.

Một mẹo thực hành khép lại mục: khi đã phát biểu được bất biến thành một câu, hãy cài luôn nó thành một phép kiểm assert chạy sau mỗi bước trong bản gỡ lỗi. Bất biến bị vi phạm ở đúng bước đầu tiên là thông tin đắt nhất mà bạn có thể có khi đi tìm lỗi (E/25).

1.9 Gốc rễ: chứng minh chương trình bắt đầu từ đâu

Ý tưởng lập luận về chương trình thay vì chỉ chạy thử nó xuất hiện sớm hơn nhiều người tưởng. Năm 1949, trong một bài viết ngắn về việc kiểm tra một chương trình lớn, Alan Turing đã trình bày gần như đầy đủ bộ khung của chương này: gắn vào từng điểm của chương trình một khẳng định về trạng thái, kiểm tra rằng mỗi bước bảo toàn các khẳng định đó, và — điểm đáng chú ý nhất — chứng minh riêng tính dừng bằng một đại lượng giảm. Nghĩa là ngay từ những năm đầu tiên của máy tính, người ta đã thấy *bất biến* và *đại lượng giảm* là hai thứ tách biệt và cùng cần thiết.

Ý tưởng đó bị bỏ quên gần hai mươi năm, cho tới khi Robert Floyd đưa ra cách gán ý nghĩa cho sơ đồ khối bằng các khẳng định (1967), và Tony Hoare biến nó thành hệ tiên đề với bộ ba tiên đề điều kiện–lệnh–hậu điều kiện (1969)[1]. Bối cảnh lịch sử giải thích vì sao chuyện này xảy ra vào đúng lúc đó: cuối thập niên 1960 là thời kỳ mà giới công nghiệp bắt đầu nói về “khủng hoảng phần mềm” — phần cứng mạnh lên nhanh, chương trình lớn lên theo, và số lỗi vượt quá khả năng kiểm soát bằng cách chạy thử. Dijkstra đẩy phản ứng ấy tới cực đoan với luận điểm rằng chương trình phải được *dẫn xuất* cùng với chứng minh của nó[2], và Gries biến nó thành sách dạy được[6].

Nhánh này về sau chia làm hai. Nhánh thực dụng thấm vào thói quen hằng ngày: khẳng định trong code, kiểm thử dựa trên tính chất, và chính lối viết bất biến bằng lời mà chương này khuyến nghị. Nhánh hình thức đi tới các công cụ kiểm chứng tự động — kiểm tra mô hình và trợ lý chứng minh — và tới thập niên 2000 đã đủ mạnh để chứng minh trọn vẹn những phần mềm thật: một trình biên dịch C có chứng minh và một nhân hệ điều hành có chứng minh là hai ví dụ được nhắc nhiều nhất. Cái giá là công sức lớn hơn viết code thông thường nhiều lần, nên nó chỉ dừng ở nơi mà một lỗi đắt tới mức đáng trả cái giá đó.

Với người luyện thuật toán, bài học từ lịch sử này rất cụ thể: bạn không cần nhánh hình thức, nhưng bạn cần đúng cái mà Turing đã dùng năm 1949 — một khẳng định ở mỗi điểm quan trọng, và một đại lượng giảm.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1949 — Turing	Kiểm tra một chương trình dài mà chạy thử không xuể.	Khẳng định tại từng điểm + đại lượng giảm cho tính dừng; bộ khung của cả chương này đã có mặt.
1967 — Floyd	Cần cách gán ý nghĩa chính xác cho sơ đồ khối.	Khẳng định gắn vào cạnh của sơ đồ; chính thức hoá ý tưởng bất biến.
1969 — Hoare	Muốn lập luận về chương trình như lập luận trong toán học.	Bộ ba $\{P\} S \{Q\}$; ngôn ngữ tiền điều kiện-hậu điều kiện dùng tới ngày nay.
Thập niên 1970 — Dijkstra, Gries	“Khủng hoảng phần mềm”: chương trình lớn hơn khả năng kiểm soát bằng chạy thử.	Chương trình và chứng minh phát triển cùng nhau; kiểm thử bị hạ xuống vai trò phát hiện lỗi, không phải bảo chứng.
Thập niên 1980–2000 — kiểm tra mô hình, trợ lý chứng minh	Hệ thống tương tranh và hệ thống trọng yếu mà lỗi có giá quá đắt.	Kiểm chứng tự động; đủ sức chứng minh trọn vẹn trình biên dịch và nhân hệ điều hành, với chi phí công sức rất lớn.

■ 1.10 Liên hệ ngang: bất biến là ý tưởng dùng chung của nhiều ngành

Đây có lẽ là chỗ liên hệ đẹp nhất trong cả phần A. Khái niệm “một đại lượng không đổi trong khi mọi thứ khác thay đổi” không phải phát minh của khoa học máy tính; nó là một trong những công cụ tư duy phổ quát nhất mà con người có, và nó xuất hiện dưới những cái tên khác nhau ở khắp nơi.

Trong vật lý, đó là các định luật bảo toàn: năng lượng, động lượng, điện tích. Ta không giải phương trình chuyển động cho từng hạt trong một vụ va chạm — ta viết ra đại lượng được bảo toàn và suy ngay ra kết quả. Đó chính xác là cách bất biến được dùng để chứng minh một thuật toán mà không phải theo dõi từng vòng lặp.

Trong lý thuyết điều khiển, đó là hàm Lyapunov: một hàm không âm của trạng thái, luôn giảm theo thời gian, dùng để chứng minh hệ hội tụ về điểm cân bằng. So sánh với mục 2 của chương này thì thấy ngay: hàm Lyapunov *chính là* đại lượng giảm, chỉ khác ở chỗ nó nhận giá trị thực và giảm liên tục thay vì nguyên và giảm rời rạc. Cùng một ý: gán một con số cho trạng thái, chứng minh con số đó đi đúng một chiều, và kết luận về hành vi dài hạn mà không phải mô phỏng.

Trong toán olympiad, bất biến và bán bất biến là một chủ đề riêng: tô màu bàn cờ, xét tính chẵn lẻ, xét tổng theo modulo — tất cả để chứng minh một cấu hình *không thể* đạt tới. Đây cũng là cách dùng mà lập trình viên hay quên: bất biến không chỉ để chứng minh thuật toán đúng, mà còn để chứng minh một việc là bất khả thi, tức là để *ngừng tìm* một lời giải không tồn tại.

■ Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Vật lý	Định luật bảo toàn năng lượng, động lượng	Cùng một chiến lược: thay vì theo dõi diễn biến chi tiết, tìm đại lượng không đổi rồi suy trực tiếp ra kết luận. Khác: bất biến vật lý do tự nhiên quy định, còn bất biến thuật toán do bạn <i>chọn</i> — chọn khéo hay vụng quyết định chứng minh dễ hay khó.
Lý thuyết điều khiển	Hàm Lyapunov: hàm không âm, giảm dọc quỹ đạo $\Rightarrow$ hệ ổn định	Chính là đại lượng giảm ở mục 2, phiên bản liên tục. Cả hai đều chứng minh “rồi sẽ tới đích” mà không cần giải ra nghiệm. Khác: hệ liên tục cần thêm điều kiện để loại trường hợp giảm mãi mà không tới nơi.
Toán olympiad	Bất biến và bán bất biến: chẵn lẻ, tô màu, tổng theo modulo	Dùng để chứng minh <i>bất khả thi</i> — công dụng hay bị bỏ quên. Trong lập trình, nó tiết kiệm hàng giờ: nhận ra cấu hình đích vi phạm một bất biến thì bạn ngừng tìm thuật toán ngay.
Kế toán	Ghi sổ kép: tổng nợ luôn bằng tổng có	Một bất biến toàn hệ thống được kiểm tra liên tục; lệch một đồng là dấu hiệu có lỗi ở đâu đó. Tương đương assert chạy ở mọi bước.
Cơ sở dữ liệu	Ràng buộc toàn vẹn và tính chất giao dịch	Giao dịch được định nghĩa đúng bằng ngôn ngữ bất biến: hệ đúng trước, có thể tạm sai ở giữa, phải đúng lại khi kết thúc — y hệt cấu trúc khởi tạo / duy trì / kết thúc.
Ước lượng trạng thái, SLAM	Ràng buộc khả quan sát: bộ lọc không được tự sinh thông tin về những hướng mà dữ liệu không quan sát được	Đây là một bất biến thật của bộ ước lượng, và vi phạm nó gây ra sai lệch có hệ thống chứ không phải nhiễu ngẫu nhiên. Cùng bài học với mục 6: bất biến vi phạm thâm lạng nguy hiểm hơn lỗi làm chương trình sập.
Trình biên dịch	Bất biến kiểu (type safety): chương trình đã qua kiểm kiểu thì không kẹt ở trạng thái vô nghĩa	Cùng khuôn ba bước, áp lên toàn ngôn ngữ thay vì một vòng lặp. Đây là ví dụ cho thấy bất biến mở rộng được từ một hàm lên cả một hệ thống.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì kiểm thử chỉ chứng minh cho những dữ liệu đã thử. Ba loại lỗi sống sót qua kiểm thử ngẫu nhiên: lỗi ở trường hợp suy biến mà bộ sinh ngẫu nhiên hầu như không tạo ra (mảng rỗng, mọi phần tử bằng nhau, $n = 1$); lỗi chỉ xuất hiện ở kích thước lớn (tràn số, quá thời gian); và lỗi ở những cấu hình xác suất thấp (một tiêu chí tham lam hỏng đúng khi có hoà). Đó là ý câu của Dijkstra: kiểm thử cho thấy sự hiện diện của lỗi, không cho thấy sự vắng mặt.
- A2. Vì viết bất biến chính là trả lời câu hỏi “sau i bước tôi giữ đúng thông tin gì”, mà đó là câu hỏi thiết kế chứ không phải câu hỏi kiểm chứng. Nếu thông tin đang giữ đủ để bước tiếp thì bạn đã có thuật toán; nếu chưa đủ thì phần còn thiếu chỉ đúng ra thứ phải bổ sung — rất thường xuyên nó trở thành một chiều mới trong trạng thái quy hoạch động. Ví dụ mở đầu ở chương A/01 là minh họa: bất biến “ cur là tổng lớn nhất của đoạn kết thúc tại j ” vừa là lời giải vừa là chứng minh.
- A3. Vì tham lam đưa ra một khẳng định mạnh hơn nhiều. Quy hoạch động chỉ khẳng định “tôi đã xét mọi khả năng phân rã”, và chứng minh chỉ cần kiểm rằng công thức truy hồi bao phủ hết các trường hợp. Tham lam khẳng định “lựa chọn cục bộ này an toàn cho *mọi* dữ liệu” — một mệnh đề toàn thể, cần lập luận đối chỗ để chứng minh, và sai chỉ với một phản ví dụ. Code ngắn hơn chính vì nó bỏ qua việc xét các khả năng, và cái giá là gánh nặng chứng minh chuyển sang bạn.
- A4. Sai ở tính dừng. Bất biến chỉ nói “nếu vòng lặp kết thúc thì kết quả đúng”; một vòng lặp vô hạn giữ mọi bất biến hoàn hảo. Cần thêm một đại lượng nguyên không âm giảm nghiêm ngặt mỗi vòng — *hi-lo* trong tìm kiếm nhị phân, số nhỏ hơn trong thuật toán Euclid. Trong đề quy, đại lượng đó là kích thước bài toán con, và chỗ hỏng phổ biến là một nhánh gọi lại chính nó với cùng kích thước.
- A5. Khi có một cơ chế kiểm chứng khác đủ mạnh và rẻ hơn — điển hình là stress test đối chiếu với lời giải vét cạn, vốn bắt được gần hết lỗi logic trên dữ liệu nhỏ. Cũng hợp lý khi thuật toán chỉ

là biến thể rất sát một kết quả đã biết, hoặc khi chi phí của một lần sai là nhỏ và phát hiện được ngay. Không hợp lý là bỏ cả hai — không chứng minh và cũng không stress test — rồi dựa vào cảm giác “nhìn có vẻ đúng”.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn leo cầu thang trong bóng tối và muốn chắc chắn rằng cuối cùng mình sẽ lên tới sân thượng. Bạn cần hai điều. Điều thứ nhất: mỗi bước chân đều đặt lên một bậc thật, không hụt — đó là bất biến, thứ luôn đúng sau mỗi bước. Điều thứ hai: số bậc còn lại giảm đi sau mỗi bước — đó là đại lượng giảm, thứ bảo đảm bạn không đi mãi. Rất nhiều người chỉ nghĩ tới điều thứ nhất và ngạc nhiên khi chương trình chạy mãi không dừng.

Chứng minh một thuật toán tham lam thì giống chuyện khác. Bạn nói “lúc nào cũng cứ chọn cái kết thúc sớm nhất là ổn”. Người ta hỏi “sao biết?”. Cách trả lời thuyết phục nhất không phải kể vài ví dụ, mà là: đưa tôi bất kỳ cách xếp tốt nhất nào của bạn, tôi sẽ sửa nó từng chút một cho giống cách của tôi, và mỗi lần sửa tôi chứng minh nó không xấu đi. Sửa xong thì hai cách giống hệt nhau, vậy cách của tôi cũng tốt nhất.

Cuối cùng, có một điều nên nhớ về kiểm thử: chạy thử giống như kiểm tra vài viên gạch trong một bức tường. Thấy chục viên tốt không có nghĩa cả tường tốt. Nhưng chỉ cần tìm ra một viên vỡ là đủ biết tường có vấn đề — nên khi nghi ngờ, hãy tìm viên vỡ ở đúng những chỗ dễ vỡ nhất: góc tường, mép, chỗ mới xây.

Một câu: Bất biến trả lời “tôi đang giữ gì”; đại lượng giảm trả lời “tôi có dừng không”; đối chỗ trả lời “vì sao chọn thế là an toàn”.

Chỗ không rút gọn được: Không có quy trình máy móc nào tìm ra bất biến đúng; nó đến từ việc nhìn ví dụ nhỏ và hỏi “sau bước này tôi thực sự biết điều gì”.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Ba việc phải kiểm của bất biến	Khởi tạo (đúng trước lần lặp đầu), duy trì (đúng trước $\Rightarrow$ đúng sau), kết thúc (bất biến + điều kiện thoát = kết luận cần).
●●● Bất biến không hứa điều gì	Không hứa vòng lặp dừng. Cần thêm đại lượng nguyên không âm giảm nghiêm ngặt mỗi vòng.
●●● Bất biến của tìm kiếm nhị phân (lower bound)	Mọi vị trí $< lo$ có giá trị $< x$; mọi vị trí $\geq hi$ có giá trị $\geq x$. Từ đó suy ra mọi chi tiết cài đặt.
●●● Lập luận đối chỗ	Lấy lời giải tối ưu bất kỳ, xét chỗ khác đầu tiên so với tham lam, chứng minh sửa cho giống tham lam không làm tệ đi; lặp lại tới khi trùng nhau.
●●● Lập luận cắt-dán	Nếu lời giải tối ưu chứa một lời giải bài con không tối ưu, thay bằng lời giải tối ưu của bài con sẽ được kết quả tốt hơn — mâu thuẫn. Đây là chứng minh cấu trúc con tối ưu.
●●● Vì sao tham lam khó chứng minh hơn quy hoạch động	Tham lam khẳng định “lựa chọn cục bộ an toàn cho mọi dữ liệu” — mệnh đề toàn thể; quy hoạch động chỉ khẳng định “đã xét mọi phân rã”.
●● Bà tiên đệ quy (Erickson)	Giả định mọi lời gọi đệ quy đã trả kết quả đúng; chỉ lo việc ghép kết quả bài con thành kết quả bài gốc. Đây chính là bước quy nạp.
●● Ba nơi phản ví dụ hay ẩn	Trường hợp suy biến; ranh giới hoà nhau của tiêu chí; và mọi chỗ lập luận của bạn phải nói “hiển nhiên”.
●● Phản ví dụ kinh điển của tham lam đối tiền	Mệnh giá 1, 3, 4 và số tiền 6: tham lam cho $4+1+1$, tối ưu là $3+3$.
●● Vì sao đường đi dài nhất không có cấu trúc con tối ưu	Ghép hai đường dài nhất của hai bài con có thể tạo đỉnh lặp, nên phép cắt-dán không hợp lệ.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Ba mức chứng minh	Một câu bằng lời cho mọi vòng lặp; lập luận đầy đủ không hình thức cho tham lam và truy hồi; hình thức hoá chỉ cho chỗ thật tinh vi.
●● Quy nạp mạnh dùng khi nào	Khi bài toán chia thành nhiều mảnh không đều, cần giả định đúng với mọi giá trị nhỏ hơn n chứ không chỉ $n - 1$.
● Tham lam luôn đi trước	Chứng minh bằng quy nạp rằng sau k bước, tham lam tốt ít nhất bằng mọi lời giải khác sau k bước.
● Khi nào bỏ chứng minh là hợp lý	Khi có stress test thay thế, khi thuật toán là biến thể sát một kết quả đã biết, hoặc khi sai thì rẻ và phát hiện ngay. Không hợp lý là bỏ cả hai.
●● Máy trạng thái và bất biến bảo toàn	Trạng thái đầu + quan hệ chuyển; vị từ đúng lúc đầu và được giữ qua mỗi bước thì đúng ở mọi trạng thái tới được. Vòng lặp, bài duyệt không gian trạng thái và bất biến olympiad là cùng một khung.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Làm sao tìm ra bất biến đúng? Nhìn vòng lặp rồi tôi vẫn không biết viết gì.

| Đi ngược: hỏi “khi vòng lặp kết thúc, tôi muốn có kết luận gì?” rồi hỏi “mệnh đề nào, cộng với điều kiện thoát, cho ra kết luận đó?”. Với tìm kiếm nhị phân, kết luận muốn có là “ lo là vị trí đầu tiên $\geq x$ ”, điều kiện thoát là $lo = hi$; mệnh đề khớp là “đáp án nằm trong $[lo, hi]$ ”. Một cách khác cũng hiệu quả: viết ra trạng thái các biến sau mỗi vòng cho một ví dụ $n = 5$ cụ thể, rồi tìm câu mô tả đúng mọi dòng của bảng đó.

2. Chứng minh thuật toán Dijkstra bằng bất biến như thế nào?

| Bất biến: với mọi đỉnh đã “chốt”, nhãn khoảng cách của nó bằng khoảng cách ngắn nhất thật. Duy trì: khi chốt đỉnh u có nhãn nhỏ nhất trong các đỉnh chưa chốt, giả sử tồn tại đường ngắn hơn tới u ; đường đó phải rời tập đã chốt tại một cạnh nào đó, tới một đỉnh v chưa chốt, và vì mọi trọng số không âm nên phần đầu của đường đó đã dài ít nhất bằng nhãn của v , mà nhãn của v không nhỏ hơn nhãn của u — mâu thuẫn. Chú ý chỗ dùng giả thiết trọng số không âm: đó chính xác là chỗ thuật toán hỏng với cạnh âm.

3. Tôi chứng minh tham lam đúng nhưng vẫn sai khi nộp. Chuyện gì xảy ra?

| Ba khả năng, theo thứ tự tần suất. Thứ nhất, chứng minh có một bước bạn nói “rõ ràng” mà thực ra không đúng — hãy đọc lại đúng những chỗ đó. Thứ hai, tiêu chí bạn cài không hoàn toàn là tiêu chí bạn chứng minh, thường ở cách phá hoà khi hai phần tử bằng nhau. Thứ ba, thuật toán đúng nhưng cài sai. Cách phân biệt nhanh: stress test với vết cạn trên $n \leq 8$. Nếu sai trên dữ liệu nhỏ thì là lỗi một hoặc ba; nếu chỉ sai ở dữ liệu lớn thì thường là tràn số hoặc quá thời gian, không phải lỗi ý tưởng.

4. Bất biến trong code (assert) có thay được chứng minh không?

| Không thay được, nhưng bổ sung rất tốt. assert kiểm bất biến trên những dữ liệu thực sự chạy qua, nên nó bắt lỗi ở đúng chỗ phát sinh thay vì để lỗi lan ra và chỉ lộ ở kết quả cuối — điều này rút ngắn thời gian gỡ lỗi rất nhiều. Cách dùng tốt: mỗi bất biến bạn viết thành chú thích, nếu kiểm được rẽ thì viết luôn thành assert. Nhớ tắt trong bản nộp nếu nó tốn thời gian.

5. Chứng minh quy hoạch động gồm những gì?

| Ba phần. Một, định nghĩa trạng thái phải rõ ràng và đủ: từ trạng thái, mọi quyết định sau này không cần biết thêm gì về quá khứ. Hai, công thức truy hồi phải *bao phủ*: mọi cách tạo ra trạng thái này đều nằm trong một nhánh của công thức. Ba, thứ tự tính phải hợp lệ: khi tính một trạng thái, mọi trạng thái nó phụ thuộc đã được tính — tức đồ thị phụ thuộc không có chu trình. Phần lớn lỗi quy hoạch động nằm ở điểm một (trạng thái thiếu chiều) hoặc điểm hai (quên một nhánh).

6. Có cần chứng minh cả những thuật toán kinh điển tôi chỉ dùng lại?

| Không cần chứng minh lại, nhưng cần biết *giả thiết* của chúng, vì đó là chỗ chúng hỏng. Dijkstra cần trọng số không âm. Tham lam Huffman cần chi phí là tổng có trọng số. Sắp xếp hai giai đoạn cần quan hệ so sánh có tính bắc cầu — một hàm so sánh không nhất quán sẽ làm `std::sort` hành xử không xác định, kể cả sắp chương trình. Quy tắc: thuộc giả thiết, không cần thuộc chứng minh.

7. Chứng minh bằng phản chứng và bằng đối chỗ khác nhau chỗ nào?

| Đối chỗ là một dạng chứng minh có tính xây dựng: nó chỉ ra cách biến lời giải tồi ưu thành lời giải tham lam, nên đọc xong bạn hiểu *vì sao* tham lam an toàn. Phản chứng chỉ nói “giả sử sai thì mâu thuẫn”, đúng nhưng ít soi sáng hơn. Trong thực hành, đối chỗ có lợi thế lớn: nếu phép đối không thực hiện được, chỗ hỏng thường chính là phản ví dụ đang cần tìm.

8. Phương pháp hàm thế năng ở chương A/03 có liên quan gì tới bất biến không?

| Rất gần nhau. Bất biến là mệnh đề đúng ở mọi thời điểm; hàm thế năng là một *đại lượng số* mà ta phát biểu bất biến về nó — “ Φ luôn không âm” — và ta lập luận qua độ biến thiên của nó. Đại lượng giảm để chứng minh tính dừng cũng cùng họ: cũng là một số nguyên với bất biến “không âm” và tính chất “giảm mỗi bước”. Ba công cụ này là ba biến tấu của cùng một ý: gán một con số cho trạng thái rồi lập luận về cách con số đó thay đổi.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Viết bất biến và chứng minh tính dừng cho sắp xếp chèn	[7]	Bất biến: đoạn $a[0..i-1]$ đã sắp và gồm đúng các phần tử ban đầu của đoạn đó.
● Viết bất biến cho thuật toán Euclid, chỉ ra đại lượng giảm	Bài tập kinh điển	Bất biến $\gcd(a, b)$ không đổi; đại lượng giảm là $\min(a, b)$.
● Tìm phản ví dụ cho tham lam “luôn lấy đồng tiền lớn nhất” với mệnh giá bất kỳ	Bài tập kinh điển	Mệnh giá 1, 3, 4 với số tiền 6. Tự tìm thêm một bộ mệnh giá khác cũng hỏng.
●● Chứng minh tham lam xếp lịch theo thời điểm kết thúc sớm nhất	[8]	Viết đầy đủ cả hai cách: đối chỗ và “tham lam luôn đi trước”.
●● Chứng minh cấu trúc con tối ưu của bài dây con chung dài nhất bằng cắt-dán	[7]	Xét ký tự cuối của hai chuỗi, chia hai trường hợp; cắt-dán ở từng trường hợp.
●● Chỉ ra chính xác chỗ chứng minh Dijkstra dùng giả thiết trọng số không âm	Chương C/17	Sau đó dựng một đồ thị có cạnh âm làm Dijkstra sai, càng nhỏ càng tốt.
●● Viết bất biến cho thuật toán hai con trỏ tìm cặp có tổng bằng x	Chương B/06	Bất biến: mọi cặp có chỉ số ngoài đoạn $[l, r]$ hiện tại đều đã bị loại một cách hợp lệ.
●●● Chứng minh thuật toán bỏ phiếu Boyer–Moore tìm phần tử quá bán	Bài tập kinh điển	Bất biến khó tìm; gợi ý: nghĩ theo kiểu “triệt tiêu từng cặp phần tử khác nhau”.
●●● Chứng minh tham lam của Kruskal đúng bằng lập luận đối chỗ	[8]	Tính chất cắt (cut property); đây là bài tập tiêu biểu cho cả họ chứng minh matroid ở chương C/15.
●●● Tìm một bài mà tham lam “có vẻ đúng” và dùng stress test để bác bỏ	Chương E/25	Mục tiêu kép: luyện viết bộ sinh test và luyện thói quen không tin cảm giác.
●●● Chứng minh tính dừng của Ford–Fulkerson với trọng số nguyên, và chỉ ra vì sao có thể không dừng với trọng số vô tỉ	Chương C/18	Đại lượng giảm chỉ tồn tại khi các bước tăng luồng bị chặn dưới bởi một hằng số dương.

Kết chương và đọc tiếp

Với chương này, phần A đã cấp đủ hai cái thước: *A/03* cho “đủ nhanh không”, chương này cho “có đúng không”. Ba công cụ đáng mang theo là bất biến (tôi đang giữ gì), đại lượng giảm (tôi có dừng không) và lập luận đối chỗ (vì sao lựa chọn này an toàn). Điểm quan trọng nhất, và cũng là điểm dễ quên nhất: cả ba đều là công cụ *thiết kế*, không chỉ công cụ kiểm chứng.

Còn một mảnh nữa của nền tảng: bộ toán mà các chương sau sẽ dùng rải rác — tổng, truy hồi, tổ hợp, xác suất. Đó là chương *A/05*, và nó là chương duy nhất trong quyển sách này không nên đọc tuần tự. Sau đó, phần B bắt đầu với câu hỏi mới: dữ liệu nên nằm ở dạng nào để thao tác tôi cần trở nên rẻ.

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Lehman, Leighton, Meyer, <i>Mathematics for Computer Science</i> , chương “State Machines”	sách giáo trình MIT, bản 2018	Nguồn của khung máy trạng thái và nguyên lý bất biến. Đọc mục 6.1–6.2 cùng bài tập Fifteen Puzzle — khoảng hai mươi trang, và là cách vào đề tốt nhất cho người viết code.
nền tảng	Hoare, “An Axiomatic Basis for Computer Programming”	Communications of the ACM, 1969	Bài mở đầu của cả ngành đặc tả và chứng minh chương trình. Ngắn, và đọc được ngay cả khi bạn không định dùng phương pháp hình thức.
tra khi cần	Gries, <i>The Science of Programming</i>	Springer, 1981	Sách dạy viết chương trình <i>cùng lúc</i> với chứng minh; đọc khi muốn kỹ năng phát biểu bất biến trở thành phản xạ.
tra khi cần	Erickson, <i>Algorithms</i>	bản điện tử miễn phí, 2019	Chương về đệ quy và quy nạp; nguồn của hình ảnh “bà tiên đệ quy” ở mục 3.

Nguồn

- [1] C. A. R. Hoare, “An Axiomatic Basis for Computer Programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [2] E. W. Dijkstra, *A Discipline of Programming*. Prentice-Hall, 1976.
- [3] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [4] J. Erickson, *Algorithms*. Independently published, 2019.
- [5] E. Lehman, F. T. Leighton, and A. R. Meyer, *Mathematics for Computer Science*. MIT, 2018, sách giáo trình, bản điện tử miễn phí.
- [6] D. Gries, *The Science of Programming*. Springer, 1981.
- [7] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [8] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.